

HotelPOS

Developer Problem Statement

Build-a-thon brief · 6 hours · Fresher Engineering Track

You are given a real product problem and a fixed window of six hours to ship something that runs. We are looking at how you think, how you scope under pressure, and how you make a system behave correctly when several parts have to agree with each other.

Read the whole brief before you write any code. The hard part of this problem is not any single screen — it is keeping four screens honest about the same live data.

1. The problem

Hotel restaurants are invisible to the tools built for them.

Generic POS systems assume one location, one brand, one team. A hotel's food-and-beverage operation breaks that assumption. A single property may run a rooftop bar, an all-day café, and a banquet kitchen at once — each with its own staff, menu, and pace. A chain multiplies that across ten cities. Today these operations are stitched together with mismatched software, paper KOTs, and WhatsApp groups, or forced into a POS that has no concept of a property hierarchy.

The cost lands on four people every service: the waiter taking orders on the floor, the kitchen reading chits and shouting across a pass, the cashier hunting for the right check and splitting bills by hand, and the manager who has no consolidated, real-time view across outlets.

2. Who you are building for

Four users, four physical contexts. Each surface you build serves one of them.

User	Role	Context that shapes the build
Aarav, 25	Floor waiter	Dinner shift at a 40-cover restaurant. On his feet 6 hours, 6 tables at once, hands always full — every second on the screen is a second not with a guest. “Take the order, send it, move. More than three taps to add a dish and I'm already behind.”
Priya, 32	KDS operator / chef	Runs the hot section on a busy Saturday night. Needs to see at a glance what's waited too long, what's next to fire, what goes out together. Stands 2 m from the screen, cannot touch it. “I don't want a dashboard. I want to see the work.”
Deepak, 28	Cashier	Bills multiple tables at close with impatient guests; split bills are the norm. Must close, take payment, and hand over a receipt in under a minute, and run the Z-report without the manager. “The worst part of my job is when the system slows me down in front of a guest.”

User	Role	Context that shapes the build
Sunita, 41	F&B manager (chain)	Oversees three properties from her phone between meetings. Needs to know if a property is underperforming, which items are moving, and whether last night reconciled — without logging into three systems. “I should see Hyderabad from Bengaluru without calling anyone.”

Four primary user roles, each with a different physical context. There may be multiple users for each role across different outlets and properties. For example, multiple waiters may take orders simultaneously while several chefs prepare them, cashiers process payments, and managers monitor operations across properties. Each surface should remain synchronized so every user sees the latest information in real time.

3. What you are building

One connected POS system with four surfaces over the same live data. These are not four products — they are four views.

Surface	Who uses it	Target form factor
Waiter app	Floor staff	Mobile (phone)
Kitchen Display (KDS)	Kitchen team	Large screen / tablet (touch-free)
Cashier terminal	Billing staff	Tablet or desktop
Manager dashboard	Property & chain managers	Web / phone

Build and demo on your laptop. The form factor tells you the layout and interaction model to design each surface for — simulate mobile or tablet by resizing a browser window. It is not the machine you code on. (Responsiveness across devices)

The system-level requirement that ties them together:

- An order placed on the waiter app must appear on the KDS in under one second.
- A table marked paid by the cashier must free itself on the waiter's floor plan immediately.
- It must work for one hotel restaurant and equally for a chain of ten properties — each outlet isolated, the brand visible across all of them.

4. The core engineering challenge

Everything above reduces to one technical problem: shared, real-time state across multiple clients with different connectivity and different update frequencies. As you build, keep asking three questions.

1. **What is the source of truth?** When the waiter app is offline and the cashier closes a table, who wins, and how does the floor plan reconcile when the waiter reconnects?
2. **How does state propagate?** WebSocket, SSE, polling — pick a transport and be ready to defend the latency and failure trade-offs.
3. **How is a tenant isolated?** Model property and outlet scoping so one outlet can never read or write another's orders, while a chain manager can read across all of them.

5. Functional requirements

Build the path through service end to end. Each surface below lists what it must do.

5.1 Waiter app

- Open a table, add items with modifiers, and fire the order to the kitchen.
- Show a live floor plan: which tables are open, occupied, sent to kitchen, or free.
- Show, per item, whether it has reached the kitchen — never leave the waiter guessing.
- Work offline and sync when connectivity returns (hotel Wi-Fi is patchy).
- Support outlet-specific menus — different outlets may expose different menu items while belonging to the same property.
- Support common item modifiers (e.g. extra sauce, no cheese).
- The waiter should always know the latest status of every ordered item without contacting the kitchen.

5.2 Kitchen Display (KDS)

- Receive new tickets in real time and show them as a queue.
- Age each ticket visually — colour or timer that escalates the longer it waits.
- Bump a ticket as complete. Bump must be operable touch-free: a large target reachable from a distance or a physical-button equivalent.
- Reflect table-side modifications to an order without the chef finding out via a returned plate.
- Display each ticket with its current status (New, Preparing, Ready, Completed) and waiting time.
- Highlight tickets that have been waiting longer than expected.

5.3 Cashier terminal

- List open tables and pull up any table's check without hunting.
- Split a bill between two or more guests.
- Apply Indian tax: CGST + SGST computed correctly on the bill.
- Take payment by cash, card, or UPI/QR, mark the table paid, and produce a receipt.
- Run an end-of-night Z-report without manager involvement.

5.4 Manager dashboard

- Live summary across at least two outlets without opening a POS terminal.
- Surface which items are moving and whether last night's close reconciled.
- Be usable on a phone-sized screen.

5.5 Permissions & approval (cross-cutting)

- Discounts and refunds require manager-level approval. Build the approval moment — the permission gate — not just the flow that follows it.

5.6 Stretch — if time allows

- A stock item flags low and blocks the corresponding menu item from being ordered.

6. Constraints

- Waiter app works offline and syncs on reconnect.
- KDS is operable without touch (physical button or large distant tap target).
- Indian tax (CGST + SGST) and payment methods (cash, card, UPI/QR) supported.

- Discounts and refunds gated behind manager approval.
- A multi-property brand has its own admin layer for the manager who never touches a terminal.

7. Out of scope today

Do not spend time on any of these — they are explicitly excluded so you can focus.

- Hotel room charge / PMS integration
- Loyalty programmes or guest profiles
- Delivery or takeaway flows
- Hardware procurement (printer drivers, card-terminal pairing)
- Multilingual UI
- Real payment-gateway integration — mock the payment step.

8. What success looks like (your demo)

By the end of the session, be ready to demonstrate the full service path live:

4. A waiter opens a table, adds an order with modifiers, and fires it — in under 30 seconds.
5. That ticket appears on the KDS in real time, ages visually, and is bumped when complete.
6. A cashier splits a bill between two guests, takes payment, and produces a receipt.
7. A manager sees a live summary across two outlets without opening a POS terminal.
8. Stretch: a low-stock item blocks its menu item from being ordered.

9. Tech choices & ground rules

- **Stack is yours.** Any language, framework, and datastore. Use what you are fastest in — we judge the result and your reasoning, not your toolchain.
- **Full-stack expected.** You own data model, API, and at least two of the four surfaces end to end, plus the real-time link between them.
- **Mock freely.** Seed data, fake auth, and a stubbed payment step are all fine. Real-time correctness matters more than breadth.
- **AI tools allowed.** Use whatever you normally use. Be ready to explain any line we point at.

10. How you will be evaluated

Area	Weight	What we look for
Real-time correctness	30%	Orders and table state stay consistent across surfaces; the sub-second propagation requirement is genuinely met, not faked in the demo.
Data model & multi-tenancy	25%	Clean property/outlet/table/order/item modelling; isolation enforced; a chain manager can read across outlets safely.
Functional coverage	20%	How much of the service path actually works end to end.
Code quality & reasoning	15%	Readable structure, sensible boundaries, and clear answers to 'why this way?'
Handling state & errors	10%	Offline sync, conflict resolution, and recovery paths — does the system stay honest when things go wrong?

Three product principles we will weigh your choices against:

- **Speed over features.** Every interaction happens under pressure, often one-handed. A working fast path beats a broad slow one.
- **Honest about state.** The waiter knows what reached the kitchen, the chef knows how long a ticket has waited, the cashier knows what is open. Make true state visible at all times.
- **Errors are expensive.** A wrong order, a missed modifier, a double charge cost real money. Prevent mistakes, and build a recovery path for when they happen anyway.

This is a working tool, not a consumer app. The people using it are professionals under pressure. Clarity is the aesthetic. Build accordingly.